The second feature of the asset pipeline is asset minification or compression. For CSS files, this is done by removing whitespace and comments. For JavaScript, more complex processes can be applied. You can choose from a set of built in options or specify your own.

The third feature of the asset pipeline is it allows coding assets via a higher-level language, with precompilation down to the actual assets. Supported languages include Sass for CSS, CoffeeScript for JavaScript, and ERB for both by default.

1.2 What is Fingerprinting and Why Should I Care?

Fingerprinting is a technique that makes the name of a file dependent on the contents of the file. When the file contents change, the filename is also changed. For content that is static or infrequently changed, this provides an easy way to tell whether two versions of a file are identical, even across different servers or deployment dates.

When a filename is unique and based on its content, HTTP headers can be set to encourage caches everywhere (whether at CDNs, at ISPs, in networking equipment, or in web browsers) to keep their own copy of the content. When the content is updated, the fingerprint will change. This will cause the remote clients to request a new copy of the content. This is generally known as *cache busting*.
The technique sprockets uses for fingerprinting is to insert a hash of the content into the name, usually at the end. For example a CSS file `global.css`

```
global-908e25f4bf641868d8683022a5b62f54.css
```

This is the strategy adopted by the Rails asset pipeline.

Rails' old strategy was to append a date-based query string to every asset linked with a built-in helper. In the source the generated code looked like this:

```
/stylesheets/global.css?1309495796
```

The query string strategy has several disadvantages:

1. **Not all caches will reliably cache content where the filename only differs by query parameters**
   Steve Souders recommends, "...avoiding a querystring for cacheable resources". He found that in this case 5-20% of requests will not be cached. Query strings in particular do not work at all with some CDNs for cache invalidation.
2. **The file name can change between nodes in multi-server environments.**
   The default query string in Rails 2.x is based on the modification time of the files. When assets are deployed to a cluster, there is no guarantee that the timestamps will be the same, resulting in different values being used depending on which server handles the request.

3. **Too much cache invalidation**
   When static assets are deployed with each new release of code, the mtime (time of last modification) of *all* these files changes, forcing all remote clients to fetch them again, even when the content of those assets has not changed.

Fingerprinting fixes these problems by avoiding query strings, and by ensuring that filenames are consistent based on their content.